Group Project: Sorting

In this project, your group will implement the following 12 sorting algorithms and analyze their performance:

- **List of algorithms:** Selection Sort, Insertion Sort, Binary Insertion Sort, Bubble Sort, Shaker Sort, Shell Sort, Heap Sort, Merge Sort, Quick Sort, Counting Sort, Radix Sort, and Flash Sort.

Please refer to the following content for the requirements.

1 Programming

1.1 Algorithms

You are asked to implement all the algorithms mentioned above (for **ascending order** only) using the **C/C++** programming language.

1.2 Experiments

The experiments necessary for this project should be conducted following the below scenario.

```
for each Data Order S1
    for each Data Size S2
        for each Sorting Algorithm S3
            1. Create an array with Data Order S1 and Data Size S2
            2. Sort the created array using the Sorting Algorithm S3, while:
                + Measuring the running time (in millisecs), and
                + Counting the number of comparisons in the algorithm
            3. Take note of S1, S2, S3, running time and number of comparisons
        end for
    end for
end for
```

1.2.1 Data Order Evaluate the sorting algorithms on different data arrangements, including `Sorted` (in ascending order), `Nearly sorted`, `Reverse sorted`, and `Randomized`.
Please refer to `DataGenerator.cpp` for more information.

1.2.2 Data Size Evaluate the sorting algorithms on data of the following sizes: `10,000`, `30,000`, `50,000`, `100,000`, `300,000`, and `500,000` elements.

1.3 Output specifications

You must compile your source code into an executable file (.exe) that can be triggered by using a command line. We will use this command to compile:

```
> g++ *.cpp -std=c++17 -o groupid[.exe]
```

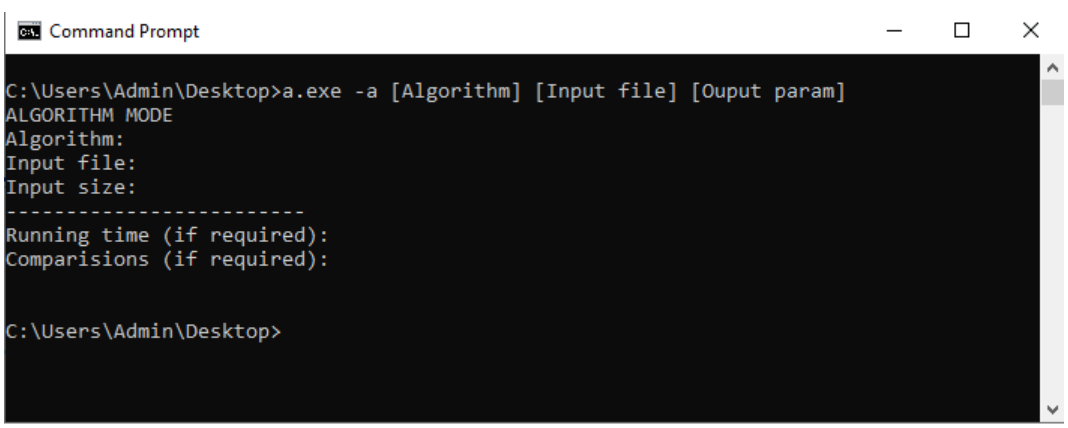
1. **Algorithm mode**: In this mode, you are asked to run a specified sorting algorithm on the input data, which is either given or generated automatically, and present the resulting running time and/or number of comparisons. General prototype:

```
> groupid[.exe] -mode Algorithm Input_file/Input_size Input_order Output_parameter

OPTIONS:
    -mode               -a algorithm mode, -c comparison mode
    Algorithm           selection-sort, insertion-sort,...
    Input_file          path to user-provided data, with format:
                        1st line: integer n (number of elements)
                        2nd line: n integers, separated by a single space
    Input_size          generated data with specified size, Integer (<= 1,000,000)
    Input_order         generated data with order: -sorted, -nsorted, -rev, -rand
    Output_parameter    -time or -comp or -both
```

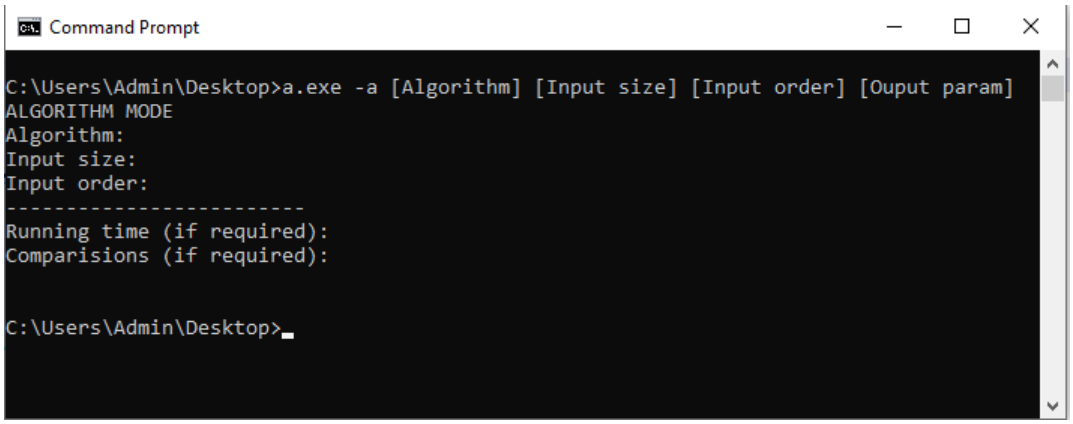
- Command 1: Run a sorting algorithm on the given input data.

```
# Prototype:
> groupid[.exe] -a Algorithm Input_file Output_parameter
# For example:
> 01[.exe] -a radix-sort input.txt -both
```



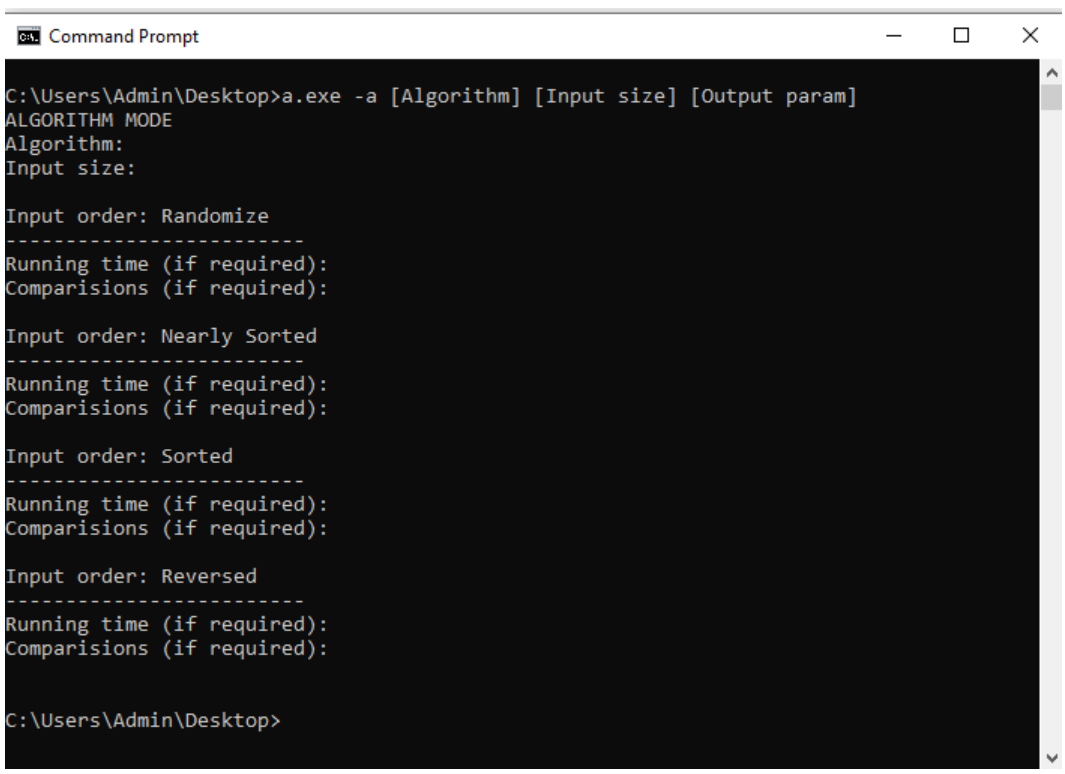
- Command 2: Run a sorting algorithm on the data generated automatically with specified size and order.

```
# Prototype:
> groupid[.exe] -a Algorithm Input_size Input_order Output_parameter
# For example:
> 01[.exe] -a selection-sort 50 -rand -time
```



- Command 3: Run a sorting algorithm on ALL data arrangements of a specified size.

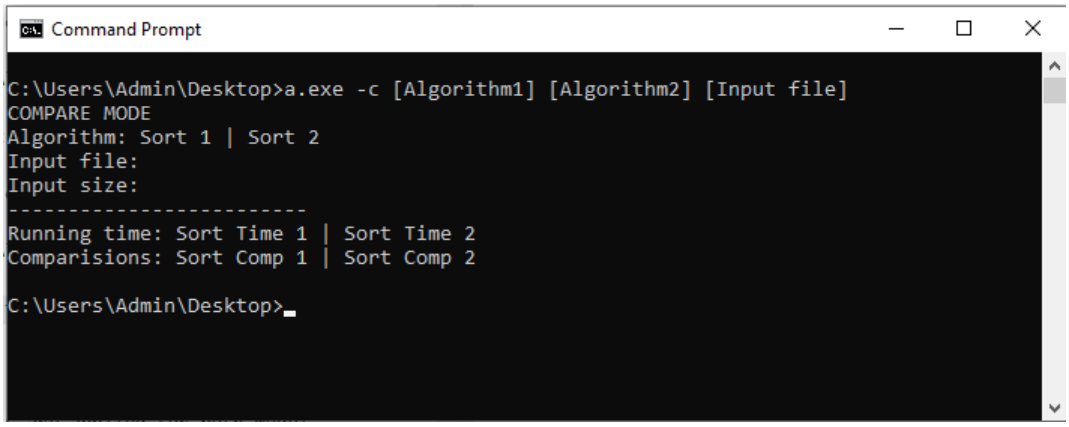
```
# Prototype:
> groupid[.exe] -a Algorithm Input_size Output_parameter
# For example:
> 01[.exe] -a quick-sort 70000 -comp
```



2. **Comparison mode:** In this mode, you have to run TWO specified sorting algorithms on the input data, which is either given or generated automatically, and present the resulting running times and/or numbers of comparisons.

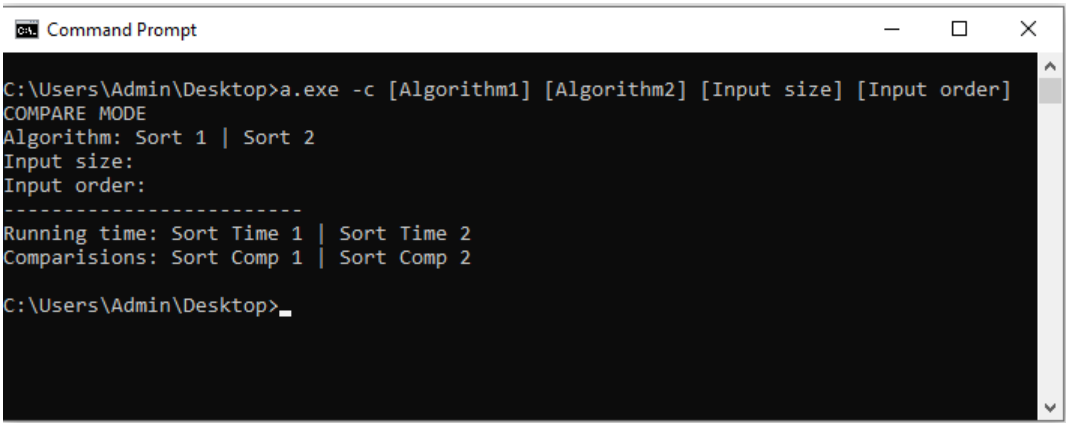
   - Command 4: Run two sorting algorithms on the given input.

```
# Prototype:
> groupid.[exe] -c Algorithm_1 Algorithm_2 Input_file
# For example:
> 01.[exe] -c heap-sort merge-sort input.txt
```



   - Command 5: Run two sorting algorithms on the data generated automatically.

```
# Prototype:
> groupid.[exe] -c Algorithm_1 Algorithm_2 Input_size Input_order
# For example:
> 01.[exe] -c quick-sort merge-sort 100000 -nsorted
```



3. **Writing files:** Besides the console output described above, you are required to write down the corresponding input(s) or output(s).

   - For Command 1 and Command 2: Write down the sorted array to the *"output.txt"* file.
   - For Command 2 and Command 5: Write down the generated input to the *"input.txt"* file.
   - For Command 3: Write down all four generated input: *"input_1.txt"* as random order data, *"input_2.txt"* as nearly sorted data, *"input_3.txt"* as sorted data, and
   - *"input_4.txt"* as reversed data.

   The file format (for both input and output files) is as follows.

   - 1st line: an integer n, indicating the number of elements in the input data
   - 2nd line: n integers, separated by a single space.

2 Report

Your report file should include the following sections:

1. **Information page** You must include your group's student information and a table showing each individual's contribution:

Student Name	ID	Primary Tasks	Contribution (%)
Nguyen Van A	25127000	• Implementation (all algorithms) • Experimental (all data types and sizes) • Command-line arguments (all 5)	80%
Le Thi C	25128989	Report (full)	15%
Tran Van D	25129998	Report (design the cover only)	1%
Dinh Hoang E	25128386	(nothing)	0%
Total			**96%**

2. **Table of Contents**

3. **Algorithm presentation**

   In this section, you present the algorithms implemented in the project:

   - Core concepts of the algorithm.
   - Step-by-step explanation, **with examples**.
   - Complexity analysis (time complexity and, if possible, space complexity).
   - Variants and improvements of the algorithm. If there are any, specifically point out the difference between the current variant and the original.

4. **Experimental results and comments**:

- You are required to organize the experimental results into FOUR tables, each representing one **Data order**. In each table, present the resulting statistics (i.e., running times or numbers of comparisons) of all sorting algorithms following a specific data arrangement. The table template is shown below:

Data order: ...						
Data size	10,000		50,000		...	
Resulting statics	Running time	Comparision	Running time	Comparision	Running time	Comparision
Sorting algorithm 1						
Sorting algorithm 2						
...						

- You are also required to make visualization by graphs.
  - There will be four LINE GRAPHs, each of which corresponds to a table of running times. In every graph, the horizontal axis is for `Data Size` and the vertical axis is for running time, as shown in Figure 1.

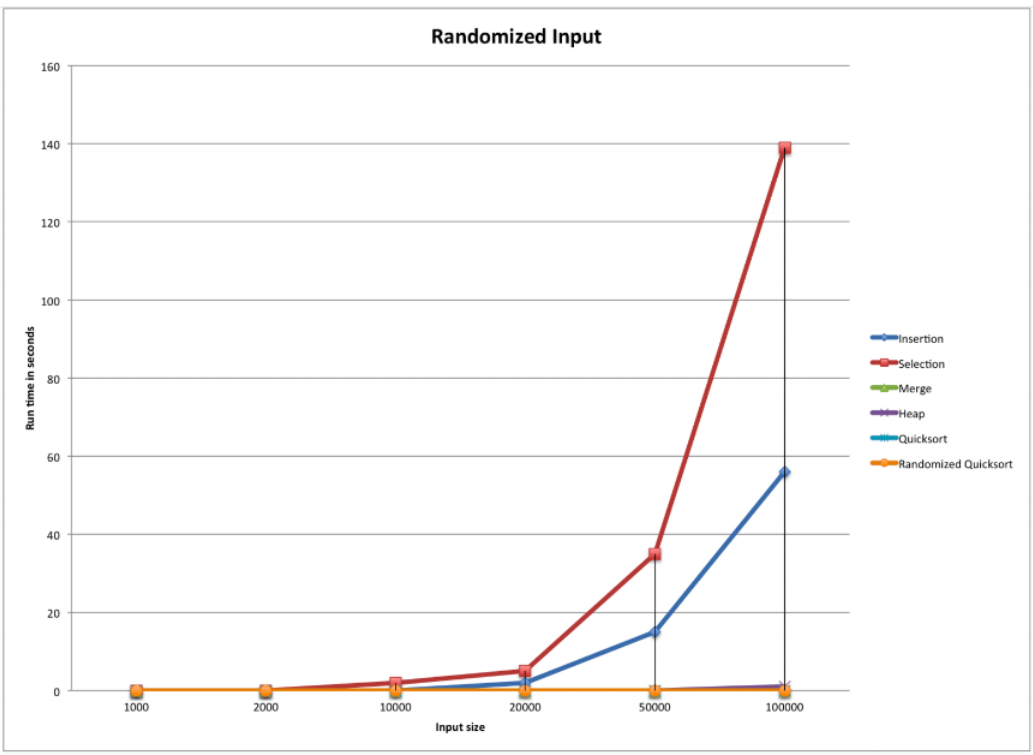


Figure 1: An example of a line graph for visualizing the algorithms' running times on randomized input data.

  - There will be four BAR CHARTs, each of which corresponds to a table of numbers of comparisons. In every graph, the horizontal axis is for `Data Size`, and the vertical axis is for the number of comparisons, as shown in Figure 2.

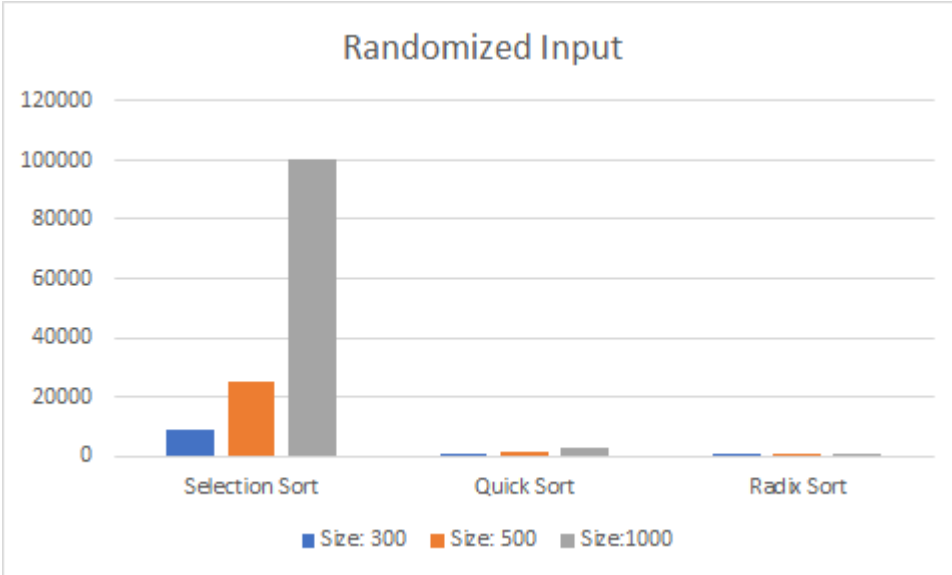


Figure 2: An example of a bar chart for visualizing the algorithms' numbers of comparisons on randomized input data.

- Make comments based on your own observations on each graph (e.g., the fastest / slowest or the most / least comparisons algorithm(s) in each case, time or comparisons acceleration of algorithms, etc.). Explain your comments.
- Make an overall comment of algorithms on all `Data Order` and all `Data Size` (the fastest / slowest algorithms overall, grouping the stable/unstable algorithms, etc.)

5. **Project organization and Programming notes**: A brief explanation of how you organized your source codes, and notes of any special libraries/data structures used.

6. **List of references**.

   1. **Author Name**, "Book Title", Reference Chapter, Publisher, Year of Publication.
   2. **Author Name**, "Article Title", Conference/Journal Name, Year of Publication.
   3. **Author Name**, "Article/Webpage Title", URL, Access Date.
   4. **Author Name**, "Slide/Lecture Note Title", [PowerPoint Slides/Lecture Notes], URL.

   Check out the IEEE Reference Guide for more details.

7. **Acknowledgments**

   Support was provided by Mr./Ms./[Relationship] ABCD in developing the code for the XYZ function, among others.

   In addition, AI tools, such as ChatGPT/Gemini/DeepSeek, assisted with drafting Section I of the report and implementing the XYZ function.

3 Submission

This is a group project for 3-5 students. If you want to form a group with fewer than 3 people, you need to get permission from the instructors. Individual assignments will not be accepted.

- Create the folder `Group ID` to include the following materials:
  - `source` folder: the project's source codes, contains only files of extensions `.cpp` and `.h` are required.
  - `Report.pdf`: Project report, only files with `.pdf` extension.
  - `README.txt`: Notes on how to run your project. DO NOT include libraries that are not part of the GNU standard.
- Compress the above folder into a file of extension **zip** and name it following your Group ID (Ex: *"1.zip"*). You can find your Group ID on the registration sheet on Moodle.
- Only **one** member representing the group submits the assignment via Moodle.

Submission that violates any regulation will get a zero.

Plagiarism and Cheating will result in a "0" (zero) for the entire course and will be subject to appropriate referral to the Management Board for further action.